Lập trình mạng · Chương 3

WebSocket: nâng cấp từ HTTP và giao tiếp hai chiều

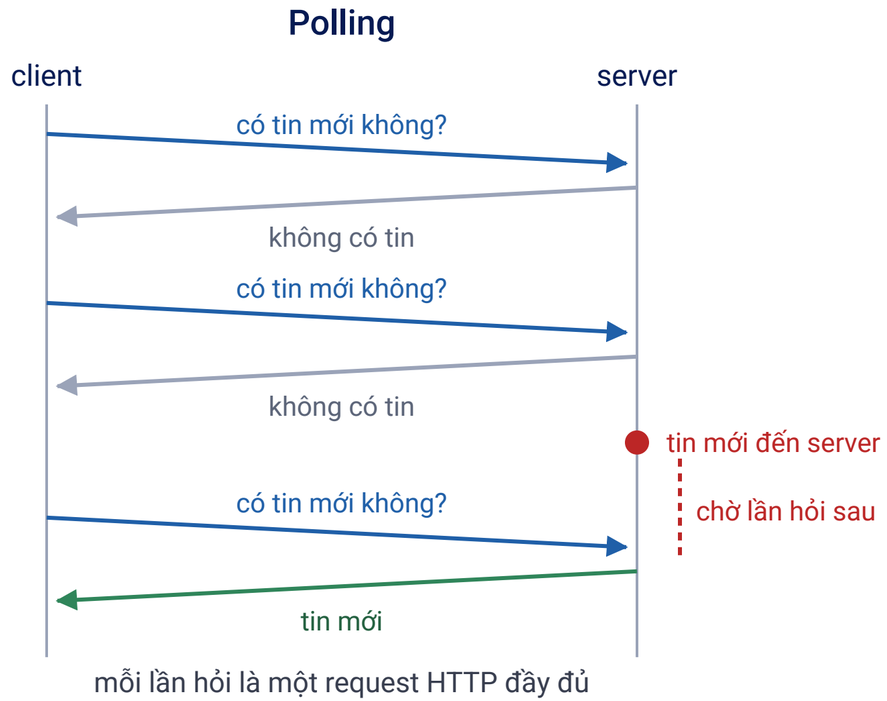
TS. Đặng Ngọc Hùng

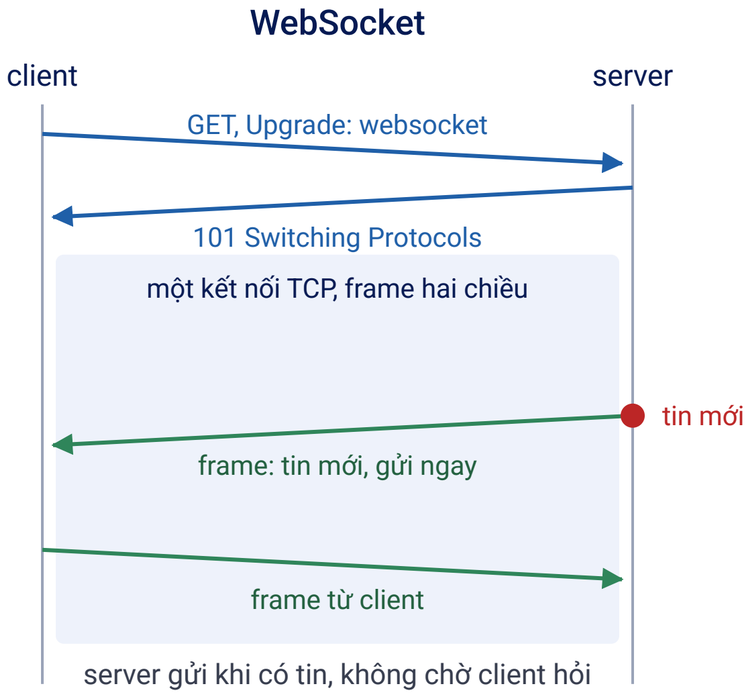
Ứng dụng web chỉ có HTTP, nhưng chat cần server gửi tin bất cứ lúc nào

1. Multicast chỉ dùng trong mạng của một tổ chức; trình duyệt không mở được socket UDP
2. Ứng dụng web chỉ có HTTP: **mô hình request–response**, server chỉ trả lời khi client hỏi
3. Chat cần server **gửi ngay khi có tin**, trên một kết nối dùng lâu dài

> **Câu hỏi của phần:** làm sao để server gửi dữ liệu cho client bất cứ lúc nào, mà vẫn bắt đầu từ HTTP?

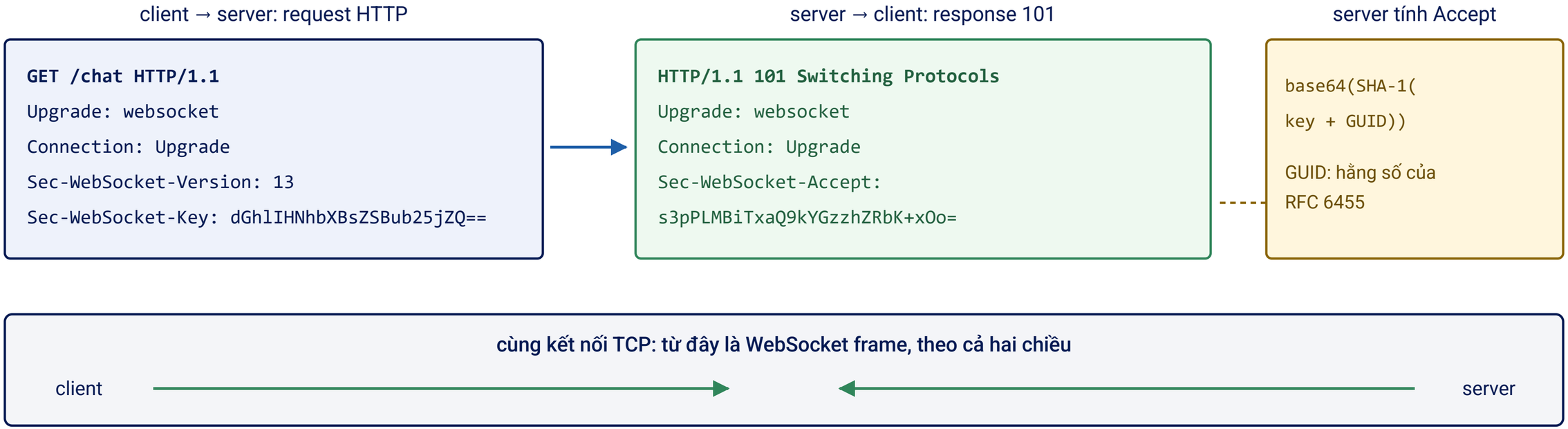
Polling hỏi định kỳ, WebSocket giữ một kết nối hai chiều





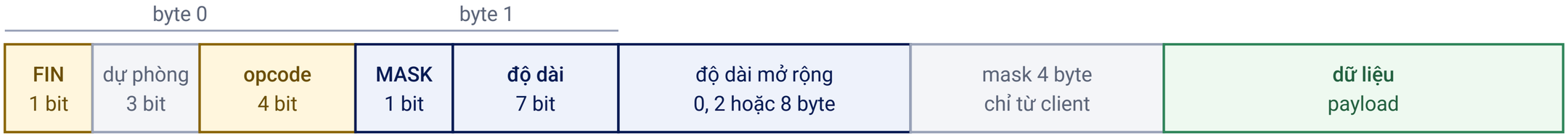
WebSocket: mở bằng một request HTTP, sau đó cùng kết nối TCP trao đổi frame theo hai chiều đồng thời (**full-duplex**).

Handshake: request có Upgrade, response 101, rồi chuyển sang frame



HTTP Upgrade: request có `Upgrade: websocket`, response `101 Switching Protocols`; sau đó là WebSocket frame trên cùng kết nối.

Mỗi WebSocket frame mang opcode và độ dài, frame từ client luôn có mask



opcode 1 text · 2 binary: message của ứng dụng

8 close · 9 ping · 10 pong: frame điều khiển

độ dài 0–125: ghi thẳng trong 7 bit

126 hoặc 127: độ dài thật ở 2 hoặc 8 byte sau

server → client, "welcome": 9 byte

```
81 07 77 65 6C 63 6F 6D 65
```

81: FIN 1, opcode 1 · 07: không mask, độ dài 7

client → server, "hello": 11 byte

```
81 85 [mask 4 byte] [5 byte đã mask]
```

85: có mask, độ dài 5

WebSocket frame: opcode cho biết loại frame, độ dài đứng trước dữ liệu (tiền tố độ dài); frame từ client luôn có mask.

Server tối giản trả 101 với Accept đúng công thức, rồi gửi frame trước

Java

```
String accept = Base64.getEncoder().encodeToString(sha1(key + GUID));  // chứng minh server hiểu WebSocket
out.write(("HTTP/1.1 101 Switching Protocols\r\n"
        + "Upgrade: websocket\r\n"
        + "Connection: Upgrade\r\n"
        + "Sec-WebSocket-Accept: " + accept + "\r\n\r\n").getBytes(US_ASCII));
Frame.write(out, Frame.TEXT, "welcome".getBytes(UTF_8));  // từ đây là frame; server gửi trước, không cần request
```

Kết quả chạy

```
> GET /chat HTTP/1.1
> Host: 127.0.0.1:9400
> User-Agent: curl/8.21.0
> Accept: */*
> Upgrade: websocket
> Sec-WebSocket-Version: 13
> Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
> Connection: Upgrade
>
< HTTP/1.1 101 Switching Protocols
< Upgrade: websocket
< Connection: Upgrade
< Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
<
```

Client dùng API WebSocket của JDK: gửi text, ping và close

Java

```
WebSocket ws = HttpClient.newHttpClient().newWebSocketBuilder()
        .buildAsync(URI.create("ws://127.0.0.1:9400/chat"), new WebSocket.Listener() {
            public CompletionStage<?> onText(WebSocket w, CharSequence data, boolean last) {
ws.sendText("hello", true).join();
ws.sendPing(ByteBuffer.wrap("still there?".getBytes(UTF_8))).join();
ws.sendClose(WebSocket.NORMAL_CLOSURE, "bye").join();
```

Kết quả chạy

```
WebSocket on port 9400
request: GET /chat HTTP/1.1
101 Switching Protocols, Sec-WebSocket-Accept: 5gbzQC9ox8l8ArdNZj5GE+LaYGk=
frame FIN=1 opcode=1 (text) masked=true length=5 "hello"
frame FIN=1 opcode=9 (ping) masked=true length=12 "still there?"
frame FIN=1 opcode=8 (close) masked=true length=5 code 1000
```

Sau handshake là frame nhị phân, và kết nối dài cần ping/pong

1. Sau handshake, dữ liệu là **frame nhị phân**: không đọc bằng `BufferedReader`, vì nó đọc trước cả byte của frame
2. Client mất mạng không gửi được frame close: dùng **ping/pong** định kỳ để phát hiện kết nối đã mất

> **Lưu ý:** WebSocket không thay TCP; nó thêm handshake HTTP và framing riêng lên trên một kết nối TCP.

Câu hỏi 1

Server có tin mới cho client trong lúc client chưa gửi gì. Với HTTP/1.1 thông thường và với WebSocket, điều gì xảy ra?

A. HTTP/1.1: server phải chờ request tiếp theo của client; WebSocket: server gửi frame ngay trên kết nối đã nâng cấp.

B. Cả hai đều gửi ngay, vì cả hai đều chạy trên TCP.

C. HTTP/1.1 gửi ngay; WebSocket phải chờ client gửi ping.

D. Cả hai đều phải chờ client hỏi; WebSocket chỉ hỏi nhanh hơn.

Câu hỏi 1

Server có tin mới cho client trong lúc client chưa gửi gì. Với HTTP/1.1 thông thường và với WebSocket, điều gì xảy ra?

A. HTTP/1.1: server phải chờ request tiếp theo của client; WebSocket: server gửi frame ngay trên kết nối đã nâng cấp.

B. Cả hai đều gửi ngay, vì cả hai đều chạy trên TCP.

C. HTTP/1.1 gửi ngay; WebSocket phải chờ client gửi ping.

D. Cả hai đều phải chờ client hỏi; WebSocket chỉ hỏi nhanh hơn.

HTTP/1.1 dùng kết nối theo mô hình request–response: mỗi response đáp lại một request, nên server không gửi được khi chưa có request. Sau HTTP Upgrade, kết nối WebSocket là full-duplex, server gửi frame bất cứ lúc nào.

Câu hỏi 2

Một ứng dụng chat có 1 000 client, mỗi client polling 2 giây một lần, trong khi mỗi phút chỉ có vài tin. Vì sao polling tốn tài nguyên hơn WebSocket?

A. Vì polling chạy trên UDP.

B. Vì WebSocket nén dữ liệu, còn HTTP thì không.

C. Vì mỗi lần polling bắt buộc mở một kết nối TCP mới.

D. Vì phần lớn request nhận về không có tin, nhưng mỗi request vẫn mang đủ header và cần server xử lý; WebSocket chỉ gửi frame khi có tin.

Câu hỏi 2

Một ứng dụng chat có 1 000 client, mỗi client polling 2 giây một lần, trong khi mỗi phút chỉ có vài tin. Vì sao polling tốn tài nguyên hơn WebSocket?

A. Vì polling chạy trên UDP.

B. Vì WebSocket nén dữ liệu, còn HTTP thì không.

C. Vì mỗi lần polling bắt buộc mở một kết nối TCP mới.

D. Vì phần lớn request nhận về không có tin, nhưng mỗi request vẫn mang đủ header và cần server xử lý; WebSocket chỉ gửi frame khi có tin.

1 000 client hỏi 2 giây một lần là 500 request mỗi giây, gần như tất cả không có tin. Polling có thể dùng lại kết nối (keep-alive), nhưng mỗi lần hỏi vẫn là một request đầy đủ.

Tổng kết

1. **HTTP** theo **mô hình request–response**; **polling** hỏi định kỳ, tốn request và tin đến chậm

2. **HTTP Upgrade**: `Upgrade: websocket`, response `101`; sau đó cùng kết nối TCP trao đổi **WebSocket frame**, **full-duplex**

3. Frame mang opcode và độ dài; **ping/pong** phát hiện kết nối đã mất

Từ khóa cần nhớ mô hình request–response · polling · HTTP Upgrade · WebSocket · WebSocket frame · full-duplex · ping/pong

Mở rộng

Chủ đề nâng cao

- Server-Sent Events: server gửi một chiều trên HTTP thường
- WebSocket qua proxy và load balancer

Bài tập mở rộng

Thực hiện handshake với WebSocket server cục bộ bằng curl; giải thích từng header trong request và response.

Tiếp theo

Bài toán ứng dụng: chat thời gian thực bằng WebSocket